Notebook 01 — Data Collection & Dataset Understanding

Decodelabs Internship | Week 1 | Task 1

I use this notebook to download the UCI Heart Disease (Cleveland) dataset, load it, assign proper column names, and develop a thorough understanding of what every column represents.

Importing libraries

```
# pandas is the main library for working with tables of data in Python.
# os lets me work with file paths in a way that works on any computer.
# requests lets me download files from the internet.

import os
import pandas as pd
import requests

print("Libraries imported successfully.")
print(f"pandas version: {pd.__version__}")
```

```
Libraries imported successfully.
pandas version: 3.0.3
```

Defining file paths

```
# The directory of this notebook
NOTEBOOK_DIR = os.getcwd()

PROJECT_ROOT = os.path.dirname(NOTEBOOK_DIR)
RAW_DATA_DIR = os.path.join(PROJECT_ROOT, "data", "raw")
PROCESSED_DATA_DIR = os.path.join(PROJECT_ROOT, "data", "processed")

# These ensure the folders exist or create them if they don't
os.makedirs(RAW_DATA_DIR, exist_ok=True)
os.makedirs(PROCESSED_DATA_DIR, exist_ok=True)

# File to be saved to the raw data
RAW_FILE_PATH = os.path.join(RAW_DATA_DIR, "heart_cleveland_raw.csv")
```

```
print(f"Project root  : {PROJECT_ROOT}")
print(f"Raw data dir  : {RAW_DATA_DIR}")
print(f"Raw file path : {RAW_FILE_PATH}")
```

```
Project root  : c:\Users\Peter\Documents\EXTRA-CURRICULA\Internship\Decodelab
Raw data dir  : c:\Users\Peter\Documents\EXTRA-CURRICULA\Internship\Decodelab
Raw file path : c:\Users\Peter\Documents\EXTRA-CURRICULA\Internship\Decodelab
```

Downloading the dataset

```
# The UCI Heart Disease (Cleveland) dataset is publicly available.
# I download it directly from the UCI Machine Learning Repository.

DATA_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-

# Column names from the UCI dataset documentation
COLUMN_NAMES = [
    "age",       # Age in years
    "sex",       # 1 = male, 0 = female
    "cp",        # Chest pain type: 1=typical angina, 2=atypical, 3=non-angi
    "trestbps",  # Resting blood pressure (mm Hg on admission)
    "chol",      # Serum cholesterol in mg/dl
    "fbs",       # Fasting blood sugar > 120 mg/dl (1=true, 0=false)
    "restecg",   # Resting ECG results (0=normal, 1=ST-T wave abnormality, 2
    "thalach",   # Maximum heart rate achieved
    "exang",     # Exercise induced angina (1=yes, 0=no)
    "oldpeak",   # ST depression induced by exercise relative to rest
    "slope",     # Slope of peak exercise ST segment (1=upsloping, 2=flat, 3
    "ca",        # Number of major vessels coloured by fluoroscopy (0–3)
    "thal",      # Thalassemia: 3=normal, 6=fixed defect, 7=reversable defec
    "target"     # Diagnosis (0=no disease, 1-4=disease present)
]

# This download the file only if it doesn't already exist
if not os.path.exists(RAW_FILE_PATH):
    print("Downloading dataset from UCI...")
    response = requests.get(DATA_URL, timeout=30)

    if response.status_code == 200:
        with open(RAW_FILE_PATH, "w") as f:
            f.write(response.text)
        print(f"Download complete. File saved to: {RAW_FILE_PATH}")
    else:
        print(f"Download failed. HTTP status: {response.status_code}")
        print("Please download manually from:")
        print(DATA_URL)
else:
    print(f"File already exists: {RAW_FILE_PATH}")
```

```
    print("Skipping download.")
```

```
Downloading dataset from UCI...
Download complete. File saved to: c:\Users\Peter\Documents\EXTRA-CURRICULA\In
```

Loading the dataset into a DataFrame

```
df = pd.read_csv(
    RAW_FILE_PATH,
    header=None,          # There is no header row in the raw file
    names=COLUMN_NAMES,   # I assign the column names manually
    na_values='?'         # Replace '?' with NaN (proper missing value marke
)

print("Dataset loaded successfully!")
print(f"Shape: {df.shape[0]} rows × {df.shape[1]} columns")
```

```
Dataset loaded successfully!
Shape: 303 rows × 14 columns
```

First look at the data

```
print("=== First 5 rows ===")
df.head()
```

```
=== First 5 rows ===
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0

```
print("=== Last 5 rows ===")
df.tail()   # useful for checking that the file loaded completely
```

```
=== Last 5 rows ===
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slo
298	45.0	1.0	1.0	110.0	264.0	0.0	0.0	132.0	0.0	1.2	
299	68.0	1.0	4.0	144.0	193.0	1.0	0.0	141.0	0.0	3.4	

300	57.0	1.0	4.0	130.0	131.0	0.0	0.0	115.0	1.0	1.2
301	57.0	0.0	2.0	130.0	236.0	0.0	2.0	174.0	0.0	0.0
302	38.0	1.0	3.0	138.0	175.0	0.0	0.0	173.0	0.0	0.0

```
rows, cols = df.shape
print(f"Dataset has {rows} rows (patients) and {cols} columns (features).")
```

```
Dataset has 303 rows (patients) and 14 columns (features).
```

Inspecting data types and non-null counts

```
# .info() shows a concise summary of the DataFrame, including:
#   - column names
#   - how many non-null values each column has (helps spot missing data quic
#   - the data type of each column (int, float, object, etc.)

print("=== DataFrame Info ===")
df.info()   # very useful for inspection.
```

```
=== DataFrame Info ===
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       303 non-null    float64
 1   sex       303 non-null    float64
 2   cp        303 non-null    float64
 3   trestbps  303 non-null    float64
 4   chol      303 non-null    float64
 5   fbs       303 non-null    float64
 6   restecg   303 non-null    float64
 7   thalach   303 non-null    float64
 8   exang     303 non-null    float64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    float64
 11  ca        299 non-null    float64
 12  thal      301 non-null    float64
 13  target    303 non-null    int64  
dtypes: float64(13), int64(1)
memory usage: 33.3 KB
```

```
# I willcheck the dtypes more directly.
# If any column is 'object' it usually means it has text or mixed types.

print("=== Column Data Types ===")
print(df.dtypes)
```

```
print(df.dtypes)
```

```
=== Column Data Types ===
age         float64
sex         float64
cp          float64
trestbps    float64
chol        float64
fbs         float64
restecg     float64
thalach     float64
exang       float64
oldpeak     float64
slope       float64
ca          float64
thal        float64
target        int64
dtype: object
```

Descriptive statistics

```
# .describe() gives me the basic statistics of the data. This gives me a qui

print("=== Descriptive Statistics ===")
df.describe().round(2)
```

```
=== Descriptive Statistics ===
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exan
count	303.00	303.00	303.00	303.00	303.00	303.00	303.00	303.00	303.0
mean	54.44	0.68	3.16	131.69	246.69	0.15	0.99	149.61	0.3
std	9.04	0.47	0.96	17.60	51.78	0.36	0.99	22.88	0.4
min	29.00	0.00	1.00	94.00	126.00	0.00	0.00	71.00	0.0
25%	48.00	0.00	3.00	120.00	211.00	0.00	0.00	133.50	0.0
50%	56.00	1.00	3.00	130.00	241.00	0.00	1.00	153.00	0.0
75%	61.00	1.00	4.00	140.00	275.00	0.00	2.00	166.00	1.0
max	77.00	1.00	4.00	200.00	564.00	1.00	2.00	202.00	1.0

Checking the target variable

```
# The 'target' column in the raw UCI data has values 0-4.
#   0 = no heart disease
#   1, 2, 3, 4 = increasing severity of heart disease
```

```
#
# In most studies, this is binarised to 0 (no disease) vs 1 (disease present
# I check the distribution here before deciding how to handle it.

print("=== Target Variable Value Counts ===")
print(df["target"].value_counts().sort_index())
print()
print("In the raw data:")
print("  0  = No heart disease")
print("  1-4 = Disease present (different severity levels)")
print()
print("I will binarise this to 0 vs 1 in Notebook 02 (cleaning step).")
```

```
=== Target Variable Value Counts ===
target
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64

In the raw data:
  0  = No heart disease
  1-4 = Disease present (different severity levels)

I will binarise this to 0 vs 1 in Notebook 02 (cleaning step).
```

Understanding what each column means

```
# I created a data dictionary — a table describing what each column means.

data_dict = {
    "Column":      ["age","sex","cp","trestbps","chol","fbs","restecg",
                    "thalach","exang","oldpeak","slope","ca","thal","target"
    "Type":        ["numeric","binary","categorical","numeric","numeric","bi
                    "categorical","numeric","binary","numeric","categorical"
                    "ordinal","categorical","target"],
    "Description": [
        "Age in years",
        "Sex (1=male, 0=female)",
        "Chest pain type (1=typical angina, 2=atypical, 3=non-anginal, 4=asy
        "Resting blood pressure on hospital admission (mm Hg)",
        "Serum cholesterol (mg/dl)",
        "Fasting blood sugar >120 mg/dl (1=true, 0=false)",
        "Resting ECG result (0=normal, 1=ST-T abnormality, 2=left ventricula
        "Maximum heart rate achieved during exercise",
        "Exercise-induced angina (1=yes, 0=no)",
        "ST depression induced by exercise relative to rest",
        "Slope of peak exercise ST segment (1=up, 2=flat, 3=down)"
```

```
            "Slope of peak exercise ST segment (1=up, 2=flat, 3=down)",
            "Number of major vessels coloured by fluoroscopy (0-3)",
            "Thalassemia (3=normal, 6=fixed defect, 7=reversable defect)",
            "Presence of heart disease (raw: 0-4; will be binarised to 0/1)"
        ],
        "Range / Values": [
            "29–77", "0, 1", "1, 2, 3, 4", "94–200", "126–564",
            "0, 1", "0, 1, 2", "71–202", "0, 1", "0.0–6.2",
            "1, 2, 3", "0, 1, 2, 3", "3, 6, 7", "0, 1, 2, 3, 4"
        ]
    }

    dict_df = pd.DataFrame(data_dict)
    print("=== Data Dictionary ===")
    dict_df
```

```
=== Data Dictionary ===
```

	Column	Type	Description	Range / Values
0	age	numeric	Age in years	29–77
1	sex	binary	Sex (1=male, 0=female)	0, 1
2	cp	categorical	Chest pain type (1=typical angina, 2=atypical,...	1, 2, 3, 4
3	trestbps	numeric	Resting blood pressure on hospital admission (...	94–200
4	chol	numeric	Serum cholesterol (mg/dl)	126–564
5	fbs	binary	Fasting blood sugar >120 mg/dl (1=true, 0=false)	0, 1
6	restecg	categorical	Resting ECG result (0=normal, 1=ST-T abnormali...	0, 1, 2
7	thalach	numeric	Maximum heart rate achieved during exercise	71–202
8	exang	binary	Exercise-induced angina (1=yes, 0=no)	0, 1
9	oldpeak	numeric	ST depression induced by exercise relative to ...	0.0–6.2
10	slope	categorical	Slope of peak exercise ST segment (1=up, 2=fla...	1, 2, 3

Checking for obvious data quality signals

```
# I look at how many missing values there are per column.
# Even though .info() showed this, I check it explicitly here.

missing_counts = df.isnull().sum()
```

```
missing_pct = (missing_counts / len(df) * 100).round(2)

missing_summary = pd.DataFrame({
    "Missing Count": missing_counts,
    "Missing %": missing_pct
})

print("=== Missing Values per Column ===")
print(missing_summary[missing_summary["Missing Count"] > 0])
print()
if missing_counts.sum() == 0:
    print("No missing values found.")
else:
    print(f"Total missing values: {missing_counts.sum()}")
    print("I will handle these properly in Notebook 02.")
```

```
=== Missing Values per Column ===
      Missing Count  Missing %
ca                4       1.32
thal              2       0.66

Total missing values: 6
I will handle these properly in Notebook 02.
```

```
# I also check for duplicate rows
# two identical rows usually indicate a data entry error or accidental dupli

n_duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {n_duplicates}")
if n_duplicates == 0:
    print("Good — no duplicate rows found.")
else:
    print("Duplicates found. I will investigate these in Notebook 02.")
```

```
Number of duplicate rows: 0
Good — no duplicate rows found.
```

Key Observations

- The dataset has **13 clinical and demographic features** and 1 target column.
- All features are **numeric** (some are binary, some categorical coded as integers).
- The target column has values **0–4**; I will binarise it to 0 vs 1 in Notebook 02.
- There are a **small number of missing values** in the `ca` and `thal` columns (marked as `?` in the raw file, now loaded as `NaN`).
- No duplicate rows were found.

Next step: Notebook 02 — Data Cleaning & Preprocessing